Describing a price line with multilayer perceptrons: plain input versus Fourier features

A plain-language but rigorous account of one experiment: fitting small neural networks to three share-price series, first with time as the raw input and then with sines and cosines of time as the input. Written for a reader with no background in computer science, statistics or higher mathematics. Every number is taken from the saved experimental output; every formula is stated in full.

1. The question

Take the daily closing price of a share over three and a half years. It is a jagged line with about 900 points. Suppose you may describe that line using only a small list of numbers, say 15, or 60, or 240, and someone else must redraw the line from your list alone. How close can the redrawn line get? And how much closer does it get each time you are allowed more numbers? That is the whole experiment. In mathematics it is called **function approximation**: finding a simple formula whose graph stays close to a complicated one.

This is deliberately **not** forecasting. Nobody is asked what the price will be tomorrow. The entire series, past and future, is on the table from the start. The earlier part of this project showed that no model we tried could forecast prices better than simply repeating yesterday's price, because day-to-day price changes are close to unpredictable. Approximation sidesteps that: it asks only how well a given kind of formula can *describe* what has already happened, which is a question about the formula, not about the market.

Why it matters. If one kind of formula describes price paths far more compactly than another, that tells us something about the formula's expressive power that forecasting experiments cannot, because forecasting results are dominated by the unpredictability of the target rather than by the model. It also tells us how 'rough' price data is, which turns out to be the deepest finding here.

2. The data

Three shares, chosen to be as different as possible: Commonwealth Bank (CBA.AX, an Australian bank, the series the original paper used), Tesla (TSLA, very volatile) and Eli Lilly (LLY, a steady upward trend). Daily closing prices from 2 January 2020 to 31 July 2023: 905 trading days for CBA, 901 for the two US shares.

Two rescalings make the three series comparable. Time is rescaled so that the first day is t = 0 and the last day is t = 1. Price is rescaled so that the lowest close in the period is 0 and the highest is 1 (this is called Min-Max scaling: p_scaled = (p - p_min) / (p_max - p_min)). After this, every series is a set of about 900 points (t_i, p_i) with both coordinates between 0 and 1. An error of 0.01 therefore means one hundredth of the share's whole price range over the period.

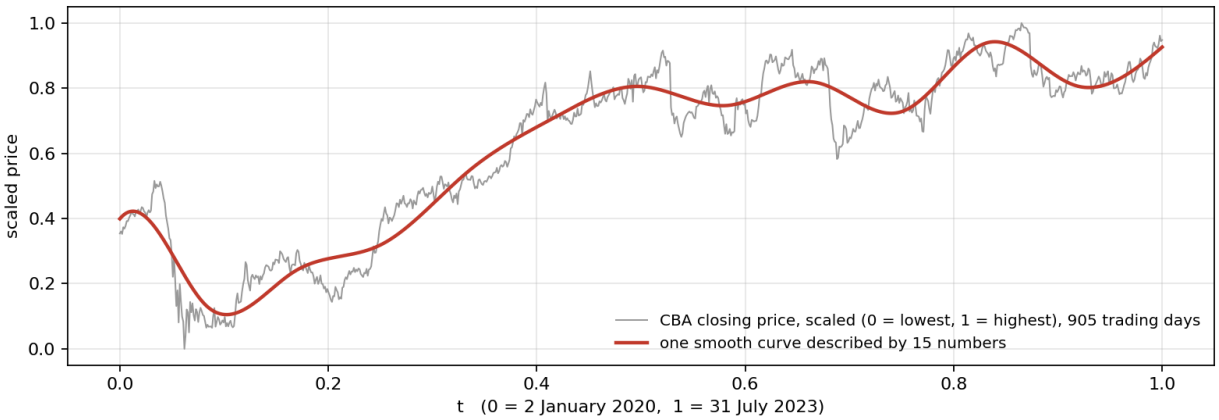


Figure 1. The CBA series after rescaling (grey), and one smooth curve that tries to follow it using only 15 numbers (red). The curve catches the broad movements and misses the fine ones. Everything in this document is about making curves like the red one, with different kinds of formula and different numbers of parameters.

3. The mathematics we need

3.1 Functions and parameters

A **function** is a rule that turns an input into an output. Here the input is a time t between 0 and 1 and the output is a price. We write f(t). A **family** of functions is a formula with blanks in it; the numbers that fill the blanks are the **parameters**. For example f(t) = a + b t + c t^2 is a family with three parameters a, b, c; choosing values for them picks one parabola out of the family. 'Describing the price line with 60 numbers' means picking the member of a 60-parameter family that sits closest to the data.

3.2 Measuring closeness: the error and the RMSE

For each day i we have the real price p_i and the curve's value f(t_i). Their difference e_i = f(t_i) - p_i is that day's **error**: positive if the curve is above the price, negative if below. One number summarises all the errors, the **root-mean-square error**:

```
RMSE = sqrt( (1/n) * sum over i of e_i^2 ) (n = number of days)
```

In words: square every error (so positives and negatives do not cancel and big errors count more), average the squares, take the square root to get back to price units. A second number, the **maximum absolute error** max |e_i|, records the single worst day. Both are reported for every fit.

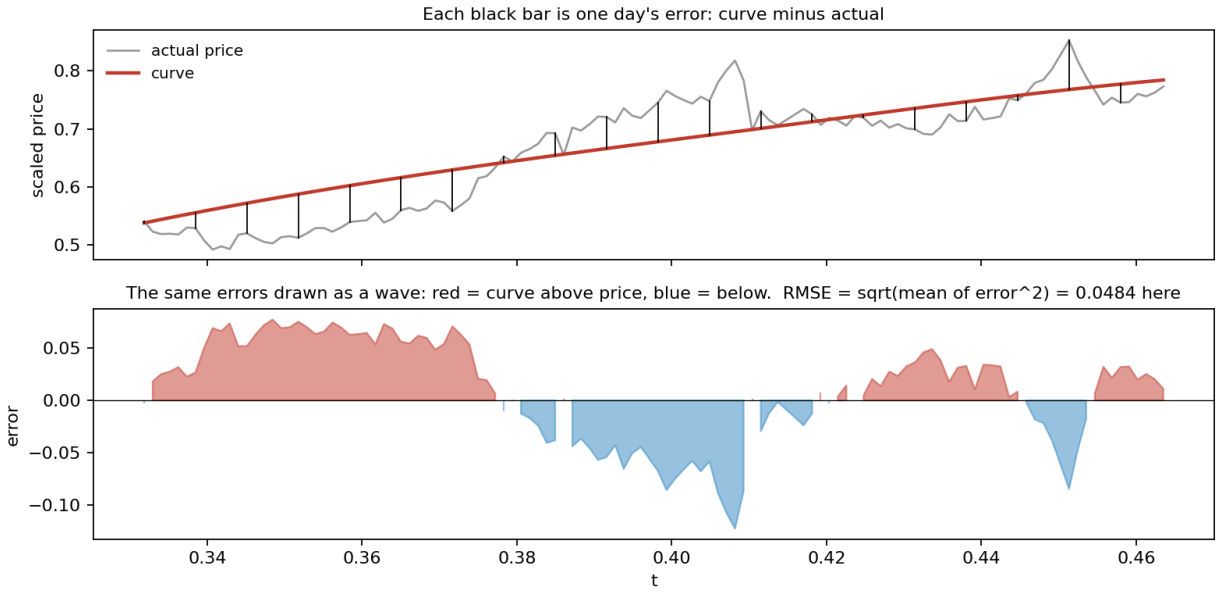


Figure 2. Top: a stretch of the price (grey) and a curve (red); each black bar is one day's error. Bottom: the same errors drawn as a wave, red where the curve is above the price, blue where below. The RMSE is the typical height of this wave. The result PDFs draw this wave for every fit, with a black envelope showing the 20-day rolling RMSE, i.e. the local typical error with the day-to-day jitter smoothed out.

3.3 'Least squares'

Among all members of a family, the one we want is the one with the smallest RMSE. Because the RMSE is built from squared errors, this is called the **least-squares fit**. For some families (splines) the least-squares fit can be computed exactly by solving a system of linear equations; for others (neural networks) it must be searched for by trial and improvement, which is what 'training' means, and the search may stop at a good-but-not-best answer.

3.4 Comparing fairly: parameter budgets

A formula with more parameters can always fit at least as well, so comparing two formulas is only meaningful at the **same number of parameters**. We therefore fixed five budgets, about 15, 30, 60, 120 and 240 parameters, set each model up to use as close to the budget as its structure allows, and recorded the exact count actually used. The plots put the true count on the horizontal axis.

3.5 Rate of improvement, and why we use logarithmic axes

The most informative quantity is not any single error but how fast the error shrinks as the parameter count N grows. For many approximation methods the error follows a **power law**, error = C * N^(-a), where a is the **rate**. Taking logarithms turns this into log(error) = log C - a * log N, a straight line whose slope is -a. So on axes where both the parameter count and the error are drawn logarithmically, a straight line means a power law and its steepness is the rate: slope -1 means doubling the parameters halves the error; slope -0.5 means doubling the parameters only cuts the error by a factor of 1.41 (the square root of 2). We fit this slope to each model's results.

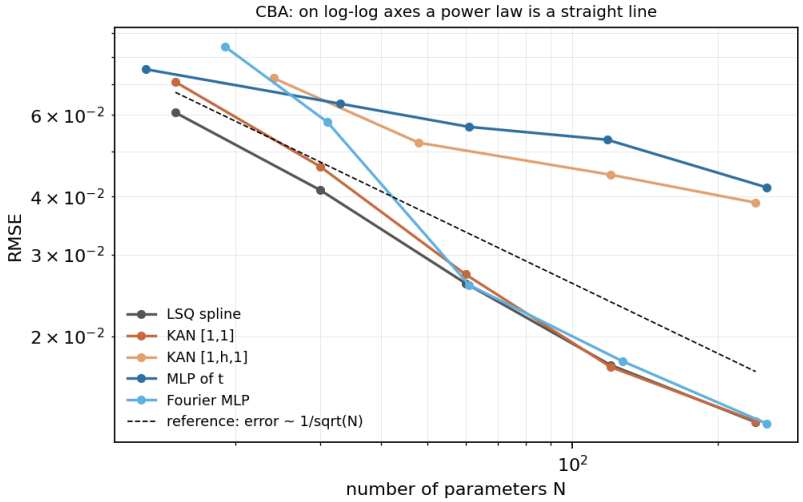


Figure 3. The CBA results on logarithmic axes. Each model's points lie roughly on a straight line; the dashed reference line has slope -0.5 (error proportional to 1 / sqrt(N)). The slope of each model is listed in the results section.

3.6 How rough is a price line? (the key background fact)

Zoom into a smooth curve and it eventually looks like a straight line. Zoom into a price line and it never does: at every magnification it is still jagged (Figure 4). Mathematicians measure this with the **Holder exponent**: a function is Holder-continuous of order alpha if |f(x) - f(y)| is at most a constant times |x - y|^alpha for all nearby x, y. A smooth curve has alpha = 1 (it is 'Lipschitz'); the path of a random walk, which is the standard model of daily prices, has alpha just below 1/2 and no higher (this is a theorem about Brownian motion; see Morters and Peres, *Brownian Motion*, Chapter 1). Classical approximation theory (de Boor, *A Practical Guide to Splines*) then says that any piecewise-smooth approximation with N pieces of a Holder-alpha function has error of order N^(-alpha). For a price path that predicts a rate of about 0.5: **doubling the parameters can only reduce the error by about 30 %, whatever the architecture**. Keep this prediction in mind; the results test it.

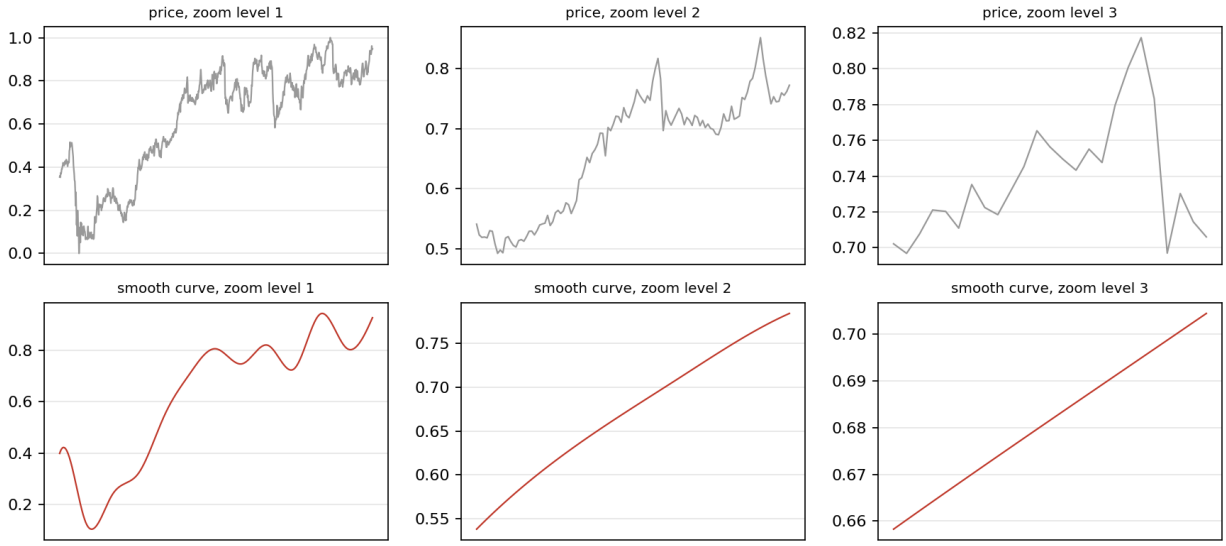


Figure 4. Top row: the CBA price line at three magnifications; it stays jagged. Bottom row: a smooth curve at the same magnifications; it straightens out. This difference is what limits every approximator in this study.

4. The two models

4.1 What an MLP is

A **multilayer perceptron** (MLP) is the standard neural network. Numbers flow left to right through layers of nodes. Each edge multiplies the number it carries by a **weight**; each node adds up its incoming numbers, adds a **bias**, and applies a fixed squashing function, here tanh(x) = (e^x - e^-x)/(e^x + e^-x), a smooth step from -1 to +1. The weights and biases are the parameters. With input t, two hidden layers of w nodes each, and one output, the formula is

```
f(t) = W3 . tanh( W2 . tanh( W1 t + b1 ) + b2 ) + b3
```

where W1 is a column of w weights, W2 a w-by-w block, W3 a row of w weights, and b1, b2, b3 the biases. Counting: w + w^2 + w weights and w + w + 1 biases, total w^2 + 4w + 1. Widths w = 2, 4, 6, 9, 14 give 13, 33, 61, 118, 253 parameters for the five budgets. The universal approximation theorem (Cybenko 1989; Hornik 1991) guarantees that some MLP can get arbitrarily close to any continuous function; it says nothing about how large it must be or whether training will find it.

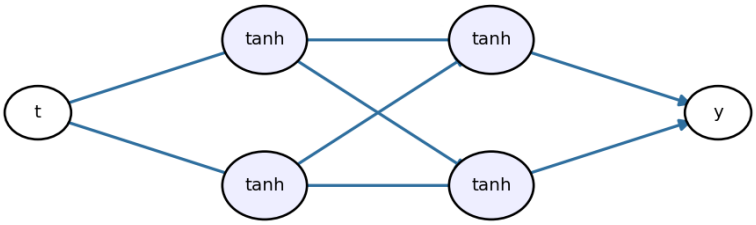


Figure 5. The MLP with w = 2, with every weight (edge) and bias (node) counted: 13 parameters.

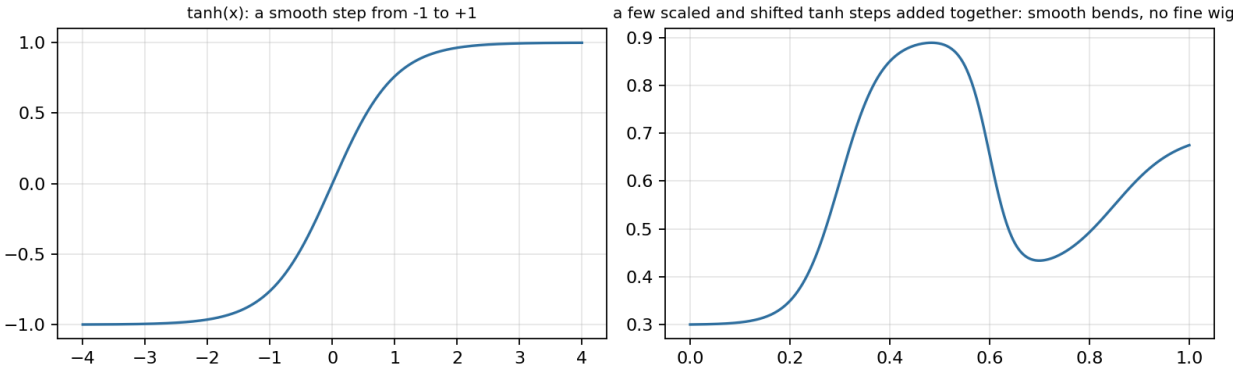


Figure 6. Left: the tanh squashing function. Right: a sum of a few scaled, shifted tanh steps. Such sums make smooth bends easily but fine wiggles only with difficulty: each wiggle needs its own steep, precisely placed step.

Training: the parameters start at random values and are adjusted step by step to reduce the RMSE (the Adam optimiser, 20,000 steps, with a decreasing step size). Because the start is random, three seeds were run. An important practical point found during this work: 4,000 steps were not enough (RMSE 0.074 on CBA at 61 parameters) and 20,000 gave 0.048; all results below use 20,000.

4.2 Model A: MLP of t

The network above with the raw time t as its single input.

4.3 Model B: Fourier-feature MLP of t

The *identical* network, but before t enters it is replaced by a list of sines and cosines of t:

```
gamma(t) = [ sin(2 pi b_1 t), ..., sin(2 pi b_m t), cos(2 pi b_1 t), ..., cos(2 pi b_m t) ]
```

with m frequencies b_1 .. b_m drawn once at random from a normal distribution with standard deviation 6 (so typical frequencies are a few cycles over the period) and then **frozen**: they are not parameters. The first layer now has 2m inputs instead of one, so the count becomes w^2 + (2m + 3) w + 1, and w is reduced to keep the budget: (m, w) = (2, 2), (2, 3), (4, 4), (6, 6), (8, 9) give 19, 31, 61, 127, 253 parameters.

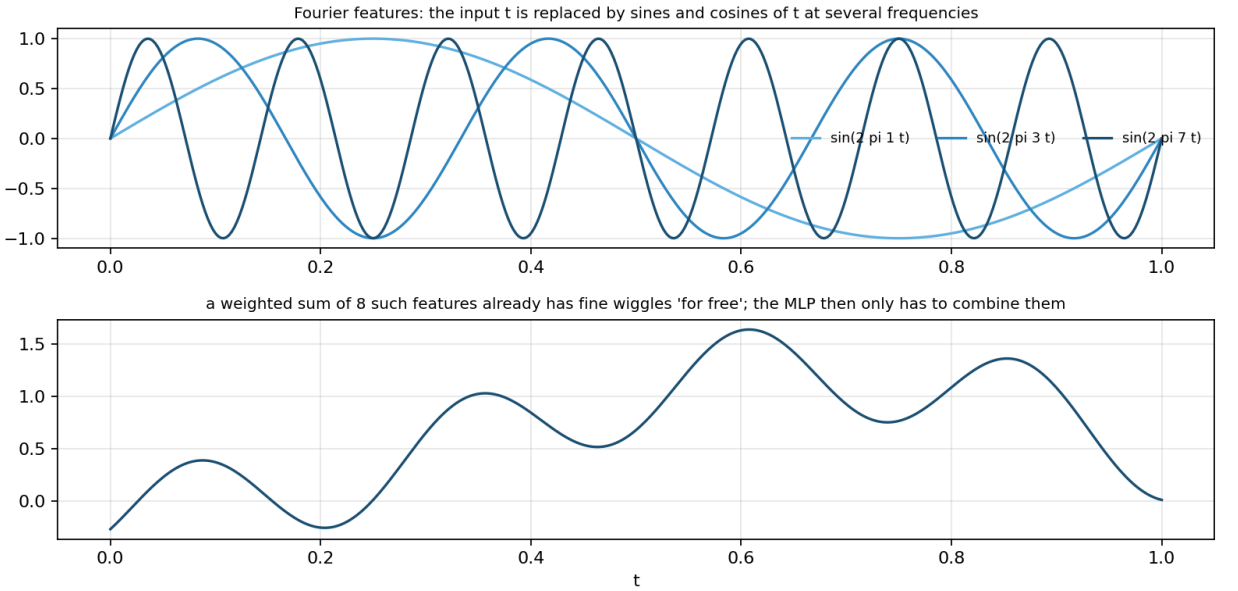


Figure 7. Top: three of the sine features at different frequencies. Bottom: a random weighted sum of eight such features already wiggles on the scale of a price line; the network only has to combine and shape them.

4.4 The one thing that differs, and why it was expected to matter

Only the input basis changes. The reason to expect a large effect is a theorem about how MLPs learn, known as **spectral bias** (Tancik et al., NeurIPS 2020, arXiv:2006.10739, building on Rahaman et al. 2019): when an MLP is trained by gradient descent on a low-dimensional input such as t, it fits the slowly varying components of the target quickly and the rapidly varying ones extremely slowly, in proportion to a rapidly decreasing function of frequency. A price line is mostly rapidly varying components. The Fourier map hands the network those components ready-made, so it no longer has to construct them from tanh steps. Tancik et al. show this turns the network's effective behaviour into that of a tunable-bandwidth interpolator.

5. The experimental setup

For each of the three shares, each model and each of the five parameter budgets, the model was fitted to the whole series by least squares (exactly for splines, by numerical optimisation for the networks). Models whose fitting involves randomness (random starting values, random frequencies) were run three times with different random seeds (0, 1, 2) and the mean and spread are reported. In total 210 fits. For every fit we saved the fitted value on every day, the RMSE, the maximum error, the parameter count, the wall-clock time and the settings.

Settings per budget for the two models in this document:

budget	15	30	60	120	240
MLP: width w / parameters	2 / 13	4 / 33	6 / 61	9 / 118	14 / 253
Fourier MLP: (m, w) / parameters	(2, 2) / 19	(2, 3) / 31	(4, 4) / 61	(6, 6) / 127	(8, 9) / 253

Software: Python 3.10, NumPy/SciPy for splines, pykan 0.2.8 for KANs, PyTorch 2.2 for MLPs; single CPU. Code: approx_study.py (fits), aggregate_approx.py (tables and figures), build_pdfs.py (result PDFs).

6. Results

share	budget	MLP of t RMSE	max err	params	Fourier MLP RMSE	max err	params
CBA.AX	15	0.0753 +- 0.0005	0.2463	13	0.0842 +- 0.0373	0.3608	19
CBA.AX	30	0.0634 +- 0.0009	0.2054	33	0.0579 +- 0.0189	0.2290	31
CBA.AX	60	0.0566 +- 0.0074	0.2032	61	0.0258 +- 0.0012	0.0966	61
CBA.AX	120	0.0531 +- 0.0078	0.2016	118	0.0177 +- 0.0016	0.0830	127
CBA.AX	240	0.0419 +- 0.0024	0.1439	253	0.0130 +- 0.0014	0.0670	253
TSLA	15	0.0868 +- 0.0075	0.3026	13	0.0985 +- 0.0398	0.3760	19
TSLA	30	0.0658 +- 0.0010	0.2490	33	0.0562 +- 0.0114	0.2467	31
TSLA	60	0.0614 +- 0.0106	0.2456	61	0.0285 +- 0.0022	0.1244	61
TSLA	120	0.0489 +- 0.0028	0.2049	118	0.0182 +- 0.0012	0.0812	127
TSLA	240	0.0404 +- 0.0041	0.1562	253	0.0143 +- 0.0013	0.0676	253
LLY	15	0.0495 +- 0.0084	0.1479	13	0.0676 +- 0.0309	0.2728	19
LLY	30	0.0400 +- 0.0008	0.1097	33	0.0431 +- 0.0175	0.1649	31
LLY	60	0.0349 +- 0.0051	0.1030	61	0.0184 +- 0.0005	0.0718	61
LLY	120	0.0352 +- 0.0027	0.1083	118	0.0124 +- 0.0015	0.0508	127
LLY	240	0.0314 +- 0.0028	0.1026	253	0.0093 +- 0.0007	0.0422	253

Table 1. RMSE (mean +- standard deviation over three seeds where the model is random) and mean maximum error, in the share's scaled price units, with the exact parameter count. For scale, the one-day-ahead naive forecast error of the same series is 0.0201 (CBA), 0.0255 (TSLA), 0.0148 (LLY).

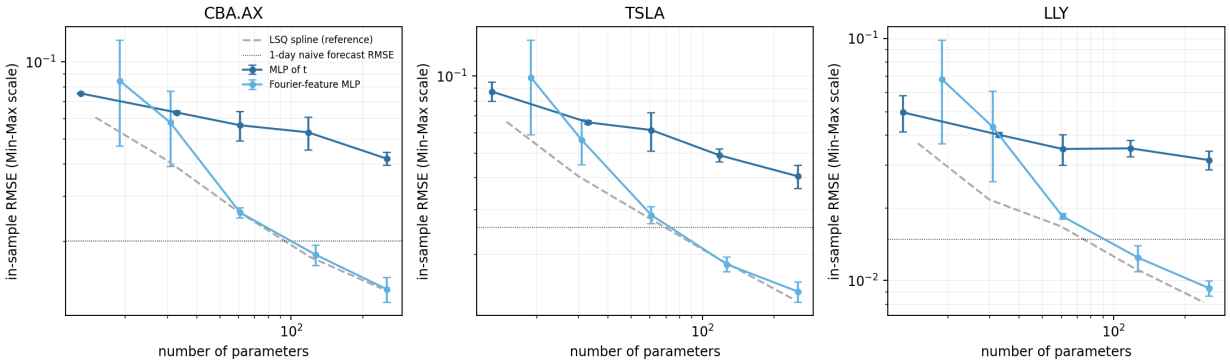


Figure 8. Error against parameter count for the two MLPs on the three shares (log-log). Grey dashed: the classical fixed-knot spline, for reference; dotted horizontal: one-day naive forecast error.

Rates of improvement (slope of log RMSE against log parameters, budgets 30 to 240): MLP -0.20, -0.25, -0.11 (mean -0.18); Fourier MLP -0.69, -0.65, -0.71 (mean -0.68), for CBA, TSLA, LLY.

7. Reading the results

The plain MLP is the weakest approximator in the whole study. On CBA it reaches only 0.0566 with 61 parameters and 0.0419 with 253, against 0.0260 and 0.0131 for a spline of the same size; its rate of improvement is -0.18, so quadrupling its size barely moves it. Figure 9 shows the characteristic failure: the curve follows the slow trend and simply does not attempt the swings. The error wave has long red and blue bands, meaning whole months are missed.

Fourier features fix it completely. With the identical network and the sine/cosine input, the error on CBA is 0.0258 at 61 parameters and 0.0130 at 253: indistinguishable from the spline (0.0260, 0.0131) and the one-edge KAN. The same holds on TSLA and LLY. Its rate, -0.68, is the steepest of any model in the study. At the two smallest budgets it is erratic (spread across seeds up to 0.04) because with only two or three random frequencies the result depends on which ones were drawn; from four frequencies upward it

is stable.

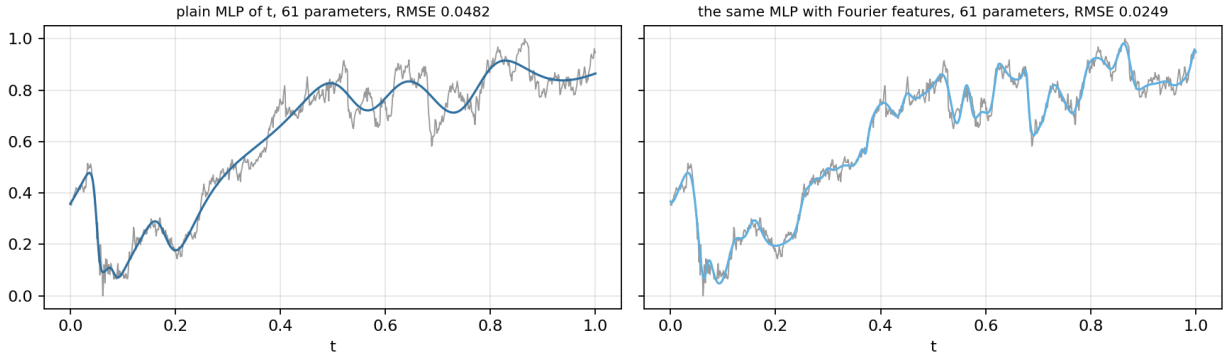


Figure 9. CBA at about 60 parameters: the plain MLP (left) draws the trend and ignores the swings; the same network with Fourier features (right) follows them. Nothing changed except the form of the input.

What the comparison establishes. The plain MLP's failure is not a shortage of capacity (the universal approximation theorem says the capacity is there) nor a shortage of training (we quintupled the training and it was still far behind). It is the spectral bias predicted by theory: tanh networks build fine detail very slowly from a raw coordinate. Supplying an oscillatory basis removes the obstacle, after which the MLP reaches the same N^(-1/2) roughness floor as every other good method, and no further. The two curves in Figure 8 are the theory's prediction drawn by the data.

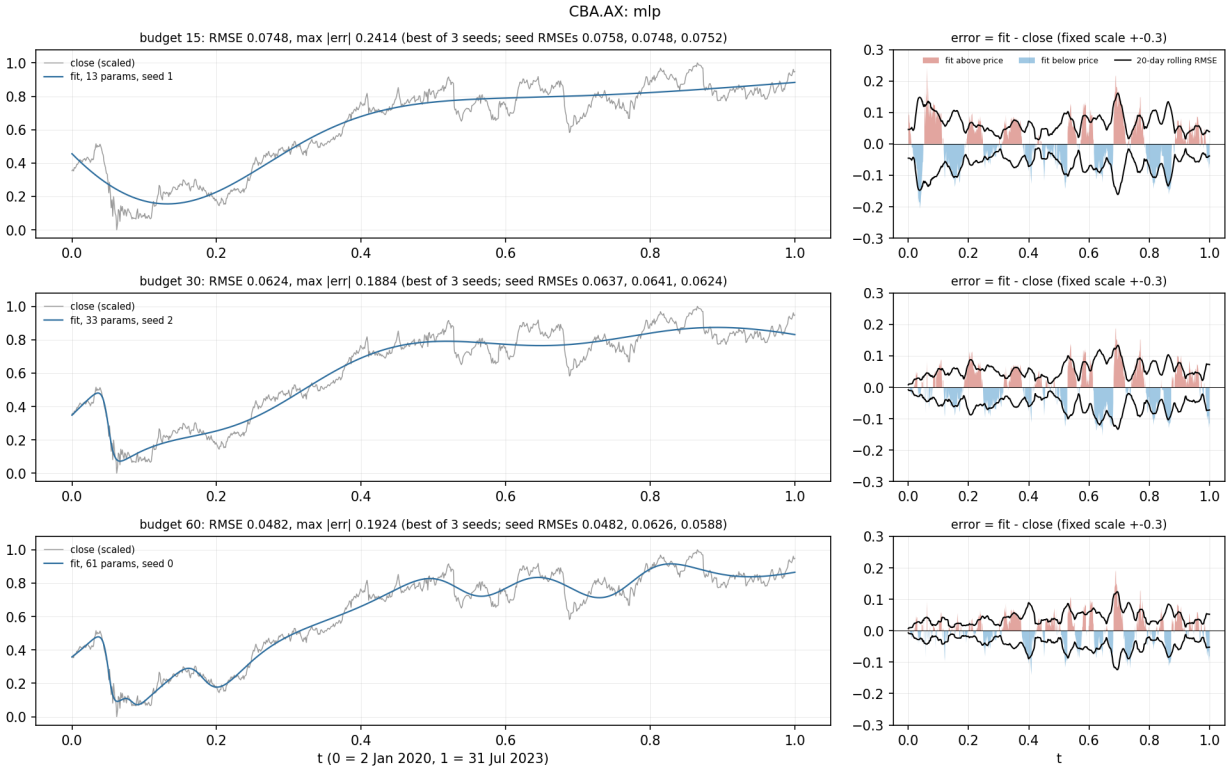


Figure 10. CBA, plain MLP at 13, 33 and 61 parameters. The rolling-RMSE envelope barely narrows as the budget rises and the error keeps its slow bands.

8. What it means

Whether 'an MLP' approximates a price line well depends entirely on what you feed it. Fed raw time, it is the worst model tested; fed sines and cosines of time, it ties the best. Any claim that a new architecture (such as a KAN) beats an MLP at approximating signals must say which MLP: against the plain one the claim is easy and uninformative, against the Fourier-feature one it is, on this data, not true. Beyond that, the Fourier MLP's convergence to the same N^(-1/2) rate as splines and KANs is further evidence that the rate belongs to the roughness of price data and not to any method: once a model can represent fine detail at all, the data decide how fast the error can fall.

9. What this experiment cannot tell you (limitations)

• Everything is in-sample: the curve is scored on the very data it was fitted to. This is the right thing for an approximation study, but it says nothing about predicting prices that have not happened yet.

• Three shares and one period. The conclusions about the *models* are consistent across all three, but the numbers themselves would differ on other shares or periods.

• Parameter counts are matched approximately (within about 10 %), not exactly.

• Three random seeds; no formal significance test was applied. Where the spread across seeds is reported, treat differences smaller than the spread as not established.

• One network shape (two hidden layers, tanh) and one Fourier bandwidth (standard deviation 6) were used; neither was tuned. A spot check with bandwidth 20 gave 0.0295 against 0.0285 at 61 parameters on CBA, so the result is not sensitive to it, but it was not optimised.

• The plain MLP is training-limited; still longer training would lower its error somewhat, but by the spectral-bias result not to the spline's level at fine scales.

10. Glossary

term	meaning
Function	a rule turning an input (here, time t) into an output (here, a price).
Parameter	a number inside a formula that is chosen to make the formula fit the data.
Least squares	choosing the parameters so that the sum of squared errors is as small as possible.
RMSE	root-mean-square error: square the daily errors, average, take the square root; the typical error size.
Min-Max scaling	rescaling so the smallest value becomes 0 and the largest becomes 1.
Power law / rate	error proportional to N^(-a); a is the rate; on log-log axes a straight line of slope -a.
Holder exponent	a measure of roughness: how fast |f(x)-f(y)| shrinks as x and y approach; 1 = smooth, 1/2 = random-walk rough.
Seed	the starting value of the random number generator; different seeds give different random starts.
MLP	multilayer perceptron: the standard neural network of weighted edges and fixed node functions.
Weight / bias	the multiplier on an edge / the constant added at a node; the MLP's parameters.
tanh	the smooth step function from -1 to +1 applied at every hidden node.
Training / gradient descent	adjusting the parameters step by step to reduce the error.
Adam	a widely used variant of gradient descent.
Fourier features	replacing the input t by sines and cosines of t at several frequencies.
Spectral bias	the proven tendency of MLPs to learn slow components first and fast components very slowly.
Universal approximation	the theorem that some MLP can approximate any continuous function to any accuracy.